CheckIn - Integration Progress Report

What was done to bring the team's separate work into one working repository.

Group 2 - IDE201 Capstone - College of Science and Technology - 8-9 October 2026

1. Summary

Before this work, each part of CheckIn lived in a different place: the database on Supabase, the frontend and a simple backend on Sanskar's laptop, and a backend design in a Word document. None of it was in the shared GitHub repository.

All three are now combined in the repo (github.com/UgayNobu/Checkin) on the dev branch, connected to the real Supabase database, and running end to end: sign-up, login and dashboards work against real data.

Part	Owner	Status
Database (Supabase + Prisma)	Dechen	Extended for the app, real data kept; one SQL file
Frontend (React + Vite)	Sanskar	Merged into repo, connected to Supabase
Backend (Express + Prisma)	Sanskar + backend teammate	Sanskar's server plus features from the design doc
Team guide	Dechen	CheckIn Team Setup Guide (.docx)

2. Step 1 - Getting the frontend running

Sanskar's project (React/Vite frontend + Express backend) was sent as a zip file. Problems fixed to run it locally:

- **Permission errors:** the zip contained a node_modules folder built on Windows. It was removed and reinstalled.
- **Port 5000 busy:** macOS AirPlay uses port 5000, so the backend runs on 5001 and the frontend points to it through frontend/.env.
- **Database mismatch:** sign-up failed with "column department does not exist", because the code expected a different version of the database.

3. Step 2 - Connecting it to our real database

Sanskar's backend was written against a mock database. The real Supabase database was kept, and the code was adapted to it instead of the other way round:

- **Prisma schema adapted:** IDs use the database's own gen_id() format (U001, M001...), evidence documents map to the existing relief_requests table, and the attendance timestamp is optional.
- **Columns added (no data lost):** users (department, programme, phone, designation, year, is_active, notification_prefs, created_at), modules (year, semester, enrolment_key, module_count), sections (section_name), class_sessions (session_type), relief_requests (leave_type, created_at).
- **Constraints and defaults:** notification category "Evidence Documents" allowed; enrollments.academic_year defaults to 2026-27.

- **Fake data replaced:** Sanskar's seed script (made-up users) was replaced with our real data: 3 programmes, 8 users, 3 modules, 3 sections, 3 sessions, 6 attendance records, 5 enrolments, 3 relief requests, 3 credit records, 4 notifications, 4 audit logs. It skips rows that already exist.
- **Test passwords:** all test users were given the password Test@1234, and a tutor account was activated for testing.

Merged to dev as Pull Request #6 ("Integrate frontend and backend; seed with real CheckIn data").

4. Step 3 - Adding the backend design doc

Our backend teammate sent a full TypeScript backend design. The team agreed to keep Sanskar's working server and only add the features missing from both his server and the database, written in plain JavaScript (no TypeScript files).

Area	Added
Sessions	List my sessions, end a session, live roster, accept/reject flagged check-ins
Check-in	Saves GPS position, distance, accuracy and flag reason (outside radius / weak GPS / no location); blocks check-in after a session ends
Relief requests	View a single request; saves original file name and review time
Tutor approvals	Pending list, approve, reject (with notification and audit log)
Reports	Audit log viewer, at-risk students + CSV, programme CSV, attendance export CSV and PDF
Alerts	Low-attendance job: notifies students below 90%, at most once a week
Notifications / profile	Unread count, mark all read, profile picture upload/remove
Security	helmet headers and rate limiting (300 requests/min)

New database columns: users (avatar_url, updated_at), class_sessions (status), attendance (submitted_lat, submitted_lng, accuracy, distance_meters, flag_reason), relief_requests (original_filename, reviewed_at).

Not added (by choice): roster-based sign-up, cookie login, email sending and Supabase Storage. They clash with how the app already works or need extra accounts. Thresholds missing from the doc use defaults (90% required, 80% at-risk, 50 m GPS accuracy, 7-day alert gap) that can be changed in .env.

All database changes and seed data were merged into one file, backend/prisma/database.sql, and run on Supabase. Merged to dev from the feature/backend-additions branch.

5. Repository now

Path	What it is
frontend/	React + Vite app (student, tutor, admin, management screens)
backend/server.js	Express API: Sanskar's routes + design-doc additions

Path	What it is
backend/prisma/schema.prisma	Prisma map of our Supabase tables
backend/prisma/database.sql	All DB changes + real seed data (safe to re-run)
backend/seed.js	npm run seed - loads the seed part of database.sql
backend/scripts/set-test-passwords.js	Sets one test password for every user
backend/API_ADDITIONS.md	List of new API routes for the frontend
.env.example files	Templates for settings (real secrets are never committed)

6. Next steps

- **Frontend:** add screens for the new routes (live roster and flagged review, end session, tutor approvals, exports, notification badge, avatar).
- **Backend:** test every new route with Postman / Thunder Client.
- **Database:** fill in session types (theory/practical) and module enrolment keys.
- **Team:** everyone sets up from the guide, works on feature branches, uses pull requests into dev, and turns on branch protection.
- **Docs:** update README (it still says Next.js and port 3000).